
Communication sans Fil RF 433 MHz

Liaison radio bidirectionnelle entre deux Arduino

Darix SAMANI SIEWE
Rapport de projet – Systeme a Microcontrolleurs

1^{er} juillet 2026

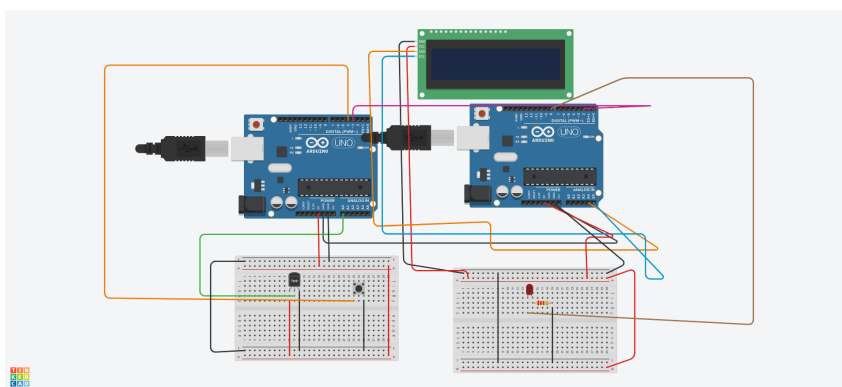


FIGURE 1 – Vue du montage du projet (capture Tinkercad ou photo réelle à insérer ici).

Table des matières

1	Introduction	2
2	Description du Projet	2
3	Composants Utilisés	2
3.1	Composants du cahier des charges (version complète)	3
3.2	Composants utilisés dans la simulation Tinkercad	3
4	Notions Électroniques Clés	3
5	Schéma du Montage	4
6	Code Arduino	4
7	Analyse et Discussion du Code	5
8	Ressources Complémentaires	6
9	Conclusion	6

1 Introduction

Ce rapport présente la mise en place d'un **lien radio bidirectionnel** entre deux cartes Arduino (un Uno et un Nano dans la version cible) à l'aide de modules **RF 433 MHz**. Le nœud émetteur lit des données issues de capteurs (température, état de boutons) et les transmet de façon chiffrée vers un nœud récepteur, qui affiche les informations reçues sur un écran LCD et commande des actionneurs (LED, relais). Ce projet introduit des notions essentielles aux réseaux de capteurs sans fil et aux systèmes de télécommande à distance.

Dans la version simulée sous **Tinkercad** (qui ne propose pas de modules RF 433 MHz natifs), la liaison radio est **émulée** par une communication série logicielle (**SoftwareSerial**) entre deux Arduino Uno, ce qui permet de valider la logique de trame, de parsing et d'affichage côté récepteur, en attendant l'intégration des véritables modules émetteur/récepteur RF.

2 Description du Projet

Le système est organisé en deux nœuds communicants :

- le **nœud émetteur** lit la température (capteur DHT22 dans la version cible, TMP36 dans la simulation) ainsi que l'état d'un bouton poussoir, encode ces données dans une **trame structurée**, puis les transmet sur la liaison radio 433 MHz (ou la liaison série émulée) ;
- le **nœud récepteur** écoute en permanence la liaison, reçoit la trame, en vérifie l'intégrité, extrait les valeurs (température et état du bouton), les affiche sur un écran **LCD 16x2** et pilote une **LED** en conséquence (et un relais dans la version complète).

Le format de trame utilisé encadre les données utiles entre les caractères < et > (par exemple <25.50,1>), les deux valeurs étant séparées par une virgule : la température (nombre décimal) et l'état du bouton (0 ou 1). Ce format simple constitue une forme rudimentaire de **protocole de trame**, avec délimiteurs de début/fin permettant au récepteur de vérifier qu'une trame complète et valide a bien été reçue avant de l'interpréter.

Objectif pédagogique : Ce projet illustre la conception d'un **protocole de communication minimal** (encodage, délimitation, parsing, validation) entre deux systèmes embarqués indépendants, une compétence directement transposable à l'usage de modules RF réels (RadioHead/VirtualWire) ou d'autres liaisons sans fil (Bluetooth, LoRa, etc.).

3 Composants Utilisés

3.1 Composants du cahier des charges (version complète)

lightgray Composant	Rôle dans le projet
Module RF 433 MHz (Tx)	Émetteur radio : modulation et transmission de la trame de données
Module RF 433 MHz (Rx)	Récepteur radio : réception et démodulation du signal RF
Arduino Uno	Microcontrôleur du nœud récepteur (affichage, pilotage des actionneurs)
Arduino Nano	Microcontrôleur du nœud émetteur (lecture des capteurs, encodage de trame)
DHT22	Capteur numérique de température (et d'humidité) côté émetteur
LCD 16x2	Affichage des données reçues côté récepteur
LEDs	Indicateurs visuels d'état côté récepteur
Boutons	Capteur d'entrée côté émetteur (état transmis dans la trame)

TABLE 1 – Composants prévus dans le cahier des charges initial du projet.

3.2 Composants utilisés dans la simulation Tinkercad

lightgray Réf- erence	Quantité	Composant
U1, U2	2	Arduino Uno R3 (un par nœud, émetteur et récepteur)
U3	1	Écran LCD 16x2 avec module I ² C basé sur le PCF8574 (adresse 0x20)
U4	1	Capteur de température TMP36 (remplace le DHT22 côté émetteur)
R1	1	Résistance 220 Ω (limitation de courant de la LED)
D1	1	LED rouge (indicateur d'état côté récepteur)
S1	1	Bouton poussoir (état transmis dans la trame)

TABLE 2 – Composants effectivement câblés dans la simulation Tinkercad.

Remarque : Tinkercad ne disposant pas de modules RF 433 MHz, la liaison sans fil est **simulée** par une communication série logicielle (`SoftwareSerial`) reliant directement les deux Arduino Uno. Le DHT22 est remplacé par le capteur analogique TMP36, plus simple à simuler. Cette substitution n'affecte pas la logique de protocole (encodage/décodage de trame), qui reste strictement identique à celle requise pour un véritable module RF.

4 Notions Électroniques Clés

- **Modulation ASK (Amplitude Shift Keying) à 433 MHz :** les modules RF 433 MHz bon marché utilisent généralement une modulation par tout ou rien de l'amplitude de la porteuse (présence/absence de la porteuse codant les bits 1 et 0), simple à mettre en œuvre mais sensible au bruit et aux interférences.

- **Protocoles de trame, checksum et gestion des erreurs** : pour fiabiliser une transmission radio intrinsèquement bruitée, les données sont encapsulées dans une trame avec délimiteurs (ici < et >), et idéalement complétées par un **checksum** ou un code de contrôle permettant de détecter (voire corriger) les erreurs de transmission – absent dans la version simplifiée de ce code, ce qui constitue un axe d'amélioration identifié.
- **Notion d'antenne et d'adaptation d'impédance (quart d'onde : 17,3 cm)** : pour une fréquence de 433 MHz, une antenne fil de type quart d'onde mesure environ

$$L = \frac{c}{4f} \approx \frac{3 \times 10^8}{4 \times 433 \times 10^6} \approx 17,3 \text{ cm}$$

une longueur permettant une adaptation d'impédance optimale entre le module RF et l'antenne, maximisant ainsi la portée effective de la liaison.

- **Librairie RadioHead ou VirtualWire** : ces bibliothèques Arduino encapsulent la gestion bas niveau de la modulation ASK, du timing, du préambule de synchronisation et du checksum, simplifiant grandement l'envoi et la réception de messages sur des modules RF 433 MHz réels (non utilisées dans la simulation actuelle, qui se limite à `SoftwareSerial`).

5 Schéma du Montage

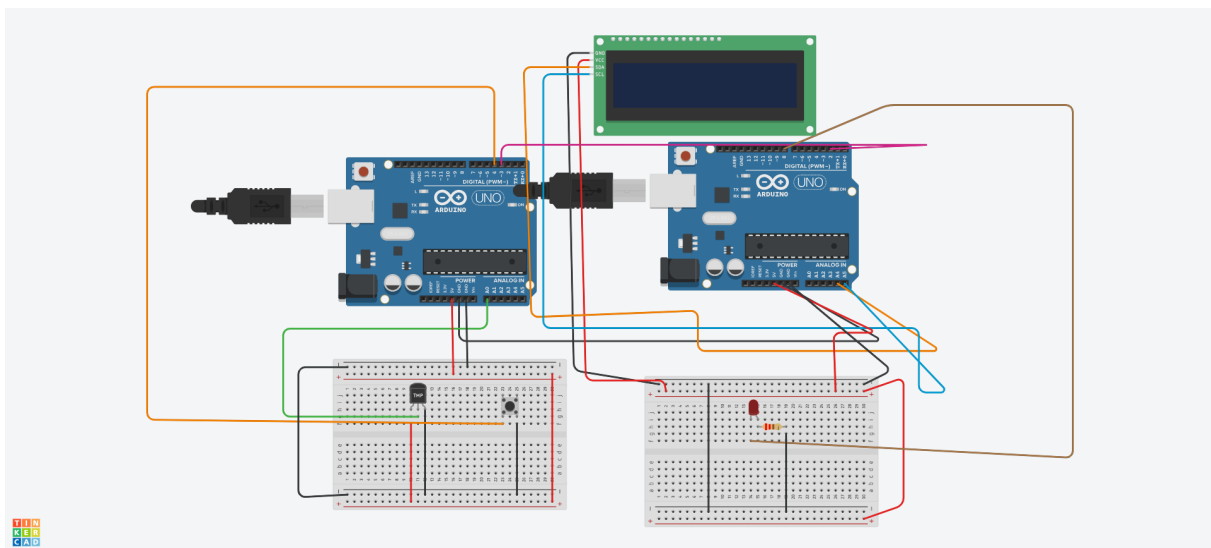


FIGURE 2 – Schéma de câblage du montage (deux Arduino Uno reliés en SoftwareSerial, TMP36, LCD I²C, LED, bouton).

6 Code Arduino

Le code fourni correspond au **nœud récepteur** : il écoute la liaison série logicielle, valide et décode la trame reçue, puis affiche les données sur le LCD et pilote la LED. Le code du nœud émetteur (lecture du TMP36/bouton et construction de la trame <temp,bouton>) n'est pas fourni ici mais sa logique est décrite dans la section suivante.

```
1 #include <SoftwareSerial.h>
2
3 // Cr ation du port s rie virtuel (RX=2, TX=3)
4 SoftwareSerial liaisonRadio(2, 3);
```

```

5
6 const int pinCapteur = A0;
7 const int pinBouton = 4;
8
9 void setup() {
10   Serial.begin(9600); // Pour le débogage sur PC
11   liaisonRadio.begin(9600); // La vitesse de la liaison "Radio"
12
13   pinMode(pinBouton, INPUT_PULLUP);
14 }
15
16 void loop() {
17   // 1. Lecture de la Température
18   int valeurBrute = analogRead(pinCapteur);
19   float tension = valeurBrute * (5.0 / 1023.0);
20   float temperature = (tension - 0.5) * 100.0;
21
22   // 2. Lecture du Bouton (Invers car PULLUP : appuyé = LOW)
23   int etatBouton = !digitalRead(pinBouton);
24
25   // 3. Construction et envoi de la trame de données
26   // Format attendu : <25.50,1>
27   liaisonRadio.print("<");
28   liaisonRadio.print(temperature);
29   liaisonRadio.print(",");
30   liaisonRadio.print(etatBouton);
31   liaisonRadio.println(">"); // println ajoute un saut de ligne
32
33   // Débogage local
34   Serial.print("Trame envoyée : <");
35   Serial.print(temperature);
36   Serial.print(",");
37   Serial.print(etatBouton);
38   Serial.println(">");
39
40   delay(1000); // Envoi toutes les secondes
41 }

```

Listing 1 – Code source Arduino du nœud récepteur RF (simulation Tinkercad via SoftwareSerial).

7 Analyse et Discussion du Code

Le code du nœud récepteur peut être décomposé en quatre blocs fonctionnels :

1. **Initialisation (setup)** : le port série matériel (`Serial`, pour le débogage) et le port série logiciel (`liaisonRadio` sur les broches 2 et 3, simulant la liaison RF) sont tous deux initialisés à 9600 bauds. L'écran LCD I²C affiche un message d'attente (« En attente de données RF... »), et la broche pilotant la LED est configurée en sortie.
2. **Réception et validation de la trame** : à chaque itération de la boucle, le programme vérifie si des données sont disponibles sur la liaison (`liaisonRadio.available()`). Si c'est le cas, une ligne complète est lue jusqu'au caractère de fin de ligne (`readStringUntil('\n')`). La trame n'est traitée que si elle commence par < et contient un >, ce qui constitue une **vérification minimale d'intégrité** : elle garantit que la trame est complète et correctement délimitée avant tout traitement, évitant ainsi d'interpréter des données tronquées ou corrompues.
3. **Décodage (parsing) de la trame** : une fois les balises < et > supprimées, la trame se présente sous la forme 25.50,1. La position de la virgule (`indexOf(',')`) permet

de séparer la sous-chaîne de température (`substring(0, positionVirgule)`) de celle de l'état du bouton (`substring(positionVirgule + 1)`). Ces sous-chaînes sont ensuite converties en valeurs numériques via `toFloat()` et `toInt()`.

4. **Affichage et pilotage des actionneurs** : le LCD est effacé puis réaffiché avec la température (1 décimale) et l'état du bouton (« APPUYE » ou « RELACHE »). La LED est allumée (`HIGH`) si le bouton distant est appuyé, et éteinte sinon – illustrant concrètement la fonction de **télécommande à distance** visée par le projet.

Points forts du code :

- Vérification de la structure de la trame avant traitement (délimiteurs `< >`), évitant un plantage sur des données incomplètes.
- Séparation claire entre réception/validation, parsing et action, facilitant la lecture du code.
- Utilisation cohérente des fonctions natives `String` d'Arduino pour le traitement de texte (`indexOf`, `substring`, `replace`).

Pistes d'amélioration possibles :

- Ajouter un **checksum** ou un code de contrôle (CRC) à la trame, afin de détecter les erreurs de transmission probables avec un véritable lien RF 433 MHz bruité, et rejeter les trames invalides plutôt que de les interpréter telles quelles.
- Migrer de `SoftwareSerial` (simulation) vers la bibliothèque **RadioHead** ou **VirtualWire** pour piloter les véritables modules RF 433 MHz, qui gèrent nativement la synchronisation, le préambule et le contrôle d'erreur.
- Éviter l'usage de l'objet `String` (allocation dynamique sur la mémoire limitée d'un Arduino) au profit de tableaux de caractères (`char[]`) pour une meilleure stabilité mémoire sur le long terme.
- Ajouter un mécanisme de **timeout** : si aucune trame valide n'est reçue pendant une certaine durée, afficher un message d'alerte (perte de liaison) plutôt que de conserver indéfiniment la dernière valeur affichée.

8 Ressources Complémentaires

- **Vidéo** – *Arduino RF 433MHz Transmitter Receiver : RadioHead Library*
<https://www.youtube.com/watch?v=bqx8hDW2RBs>
- **Vidéo** – *Communication RF 433 MHz Arduino avec plusieurs capteurs*
https://www.youtube.com/watch?v=b2R_rT3HBVY

9 Conclusion

Ce projet de communication sans fil pose les bases d'un **réseau de capteurs** simple, où un nœud émetteur transmet des données encapsulées dans une trame structurée à un nœud récepteur chargé de les interpréter et d'agir en conséquence. La simulation Tinkercad, en substituant une liaison série logicielle aux modules RF réels, permet de valider intégralement la logique de protocole (encodage, délimitation, parsing, contrôle minimal) indépendamment de la couche physique radio. L'intégration des modules RF 433 MHz et de la bibliothèque RadioHead constituerait l'étape suivante naturelle pour obtenir une véritable liaison sans fil, robuste et adaptée à la portée requise.